HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1
BỘ MÔN PYTHON



BÀI TẬP LỚN

HỌC PHẦN: LẬP TRÌNH PYTHON

Đề tài: HRAG – Hierarchical Agentic RAG, hệ thống hỏi đáp tài liệu dài

Giảng viên hướng dẫn:
Lớp học phần:
Nhóm thực hiện: 1
Danh sách thành viên:
Đinh Tuấn Quang - B24DCKH111
Lê Anh Tú - B24DCKH128
Nguyễn Duy Long – B24DCKH076

HÀ NỘI, THÁNG 10/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 4
1.1. Bối cảnh và bài toán 4
1.2. Mục tiêu 4
1.3. Phạm vi và dữ liệu 5
1.4. Phân công công việc 5
1.5. Khai báo sử dụng công cụ AI 5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 6
2.1. Naive RAG 6
2.2. RAPTOR 6
2.3. Corrective RAG (CRAG) 6
2.4. Agentic RAG và LangGraph 6
2.5. LLM-as-judge 6
CHƯƠNG 3. KIẾN TRÚC TỔNG THỂ 7
3.1. Các thành phần 7
3.2. Cấu trúc mã nguồn 8
3.3. Luồng một lượt hỏi đáp 8
CHƯƠNG 4. LẬP CHỈ MỤC: CÂY HEADING PHÂN CẤP 9
4.1. Ý tưởng 9
4.2. Quy trình xây cây 10
4.3. Chi phí lập chỉ mục 10
4.4. So sánh với RAPTOR 11
CHƯƠNG 5. AGENT TRUY HỒI 12
5.1. Đồ thị cha: phân rã và gộp khía cạnh 12
5.2. Đồ thị con: vòng truy hồi có sửa lỗi 12
5.3. Các công cụ tìm kiếm 14
5.4. Chấm điểm đoạn và đánh giá ngữ cảnh 14
5.5. Mô hình ngôn ngữ và giám sát 14
CHƯƠNG 6. ĐÁNH GIÁ 16
6.1. Phương pháp 16
6.2. Các hệ thống so sánh 16
6.3. Kết quả 17
6.4. Tiến trình cải thiện 18
6.5. Chi phí và độ trễ 18

CHƯƠNG 7. BACKEND 20
CHƯƠNG 8. CƠ SỞ DỮ LIỆU 21
CHƯƠNG 9. BỘ NHỚ HỘI THOẠI 23
CHƯƠNG 10. GIAO DIỆN STREAMLIT 24
10.1. Các chức năng chính 24
10.2. Cách giao diện tương tác với các phần khác 27
CHƯƠNG 11. CÔNG NGHỆ VÀ THƯ VIỆN 29
CHƯƠNG 12. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 30
12.1. Kết quả đạt được 30
12.2. Hạn chế 30
12.3. Hướng phát triển 30
TÀI LIỆU THAM KHẢO 31

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và bài toán

Các mô hình ngôn ngữ lớn (LLM) trả lời tốt các câu hỏi tổng quát nhưng không biết nội dung riêng của một tài liệu cụ thể và dễ bịa thông tin. Retrieval-Augmented Generation (RAG) khắc phục điều này bằng cách tìm các đoạn liên quan trong tài liệu rồi đưa vào ngữ cảnh của LLM. Với các tài liệu dài và có cấu trúc như giáo trình, luật hay báo cáo kỹ thuật, cách làm phổ biến (naive RAG: cắt văn bản thành các đoạn có kích thước cố định, nhúng vector, lấy top-k đoạn gần nhất) bộc lộ nhiều hạn chế.

- Cắt cố định phá vỡ cấu trúc chương/mục, một khái niệm bị chia vào nhiều đoạn rời nhau.
- Câu hỏi tổng hợp hoặc so sánh nhiều khái niệm cần thông tin ở nhiều vị trí khác nhau, một lần tìm top-k không đủ.
- Không có cơ chế tự kiểm tra: nếu đoạn tìm được sai hoặc thiếu thì LLM vẫn phải trả lời.
- Không có trí nhớ hội thoại nên câu hỏi nối tiếp như "còn trường hợp thứ hai thì sao?" không thể tìm kiếm được.

Đề tài **HRAG (Hierarchical Agentic RAG)** xây dựng một hệ thống hỏi đáp trên tài liệu dài theo hướng khác: lập chỉ mục dạng cây phân cấp theo heading, dùng một agent LangGraph tự quyết định công cụ tìm kiếm, tự chấm điểm và tự đánh giá độ đủ của ngữ cảnh, đồng thời có backend, cơ sở dữ liệu, bộ nhớ hội thoại và giao diện web hoàn chỉnh.

1.2. Mục tiêu

- Xây dựng chỉ mục cây phân cấp giữ nguyên cấu trúc tài liệu và cho phép truy ngược về dòng nguồn.
- Xây dựng agent truy hồi có sửa lỗi (Corrective RAG) với phân rã câu hỏi thành nhiều khía cạnh chạy song song.
- Xây dựng bộ đánh giá tự động bằng LLM-as-judge và so sánh với các phương pháp nền (naive RAG, RAPTOR).
- Xây dựng tầng services dùng chung, cơ sở dữ liệu PostgreSQL, bộ nhớ ngắn hạn cho hội thoại nhiều lượt và giao diện Streamlit.

1.3. Phạm vi và dữ liệu

Dữ liệu thử nghiệm là giáo trình "Pháp luật đại cương" (khoảng 534 nghìn ký tự, 5.491 dòng). Bộ đánh giá gồm 30 câu hỏi tiếng Việt kèm đáp án tham chiếu. Hệ thống tập trung vào hỏi đáp có trích dẫn trên tài liệu dài.

1.4. Phân công công việc

Bảng 1. Phân công công việc của nhóm

Thành viên	Phụ trách
Đinh Tuấn Quang	Agent, truy hồi thông tin và đánh giá hệ thống
Nguyễn Duy Long	Backend, cơ sở dữ liệu và bộ nhớ hội thoại
Lê Anh Tú	Giao diện người dùng Streamlit

1.5. Khai báo sử dụng công cụ AI

Trong quá trình thực hiện bài tập, nhóm có sử dụng công cụ AI là trợ lý lập trình Claude Code (Anthropic). Công cụ này được dùng cho các nhiệm vụ sau:

- Khảo sát mã nguồn của dự án để tổng hợp kiến trúc, luồng xử lý và danh sách thư viện phục vụ viết báo cáo.
- Vẽ các sơ đồ minh hoạ (kiến trúc, đồ thị agent, quy trình lập chỉ mục, sơ đồ quan hệ cơ sở dữ liệu) và biểu đồ so sánh kết quả.
- Chạy ứng dụng và chụp ảnh màn hình giao diện để đưa vào báo cáo.
- Soạn thảo và định dạng văn bản báo cáo, xuất file Word và PDF.

Các nhiệm vụ trên đều do thành viên nhóm giao và kiểm tra lại. Số liệu đánh giá trong chương 6 là kết quả các lần chạy thực nghiệm của nhóm, AI chỉ tổng hợp và trình bày. Ngoài ra, các mô hình ngôn ngữ như Gemini hay DeepSeek xuất hiện trong báo cáo là thành phần của hệ thống được xây dựng, không thuộc phạm vi công cụ hỗ trợ khai báo ở mục này.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Naive RAG

Naive RAG gồm hai giai đoạn. Giai đoạn lập chỉ mục cắt tài liệu thành các đoạn (chunk) có độ dài cố định, tính vector nhúng cho từng đoạn bằng mô hình embedding và lưu vào kho vector. Giai đoạn truy vấn nhúng câu hỏi, lấy k đoạn có độ tương đồng cosine cao nhất rồi đưa chúng cùng câu hỏi vào LLM để sinh câu trả lời. Ưu điểm là đơn giản, nhanh; nhược điểm là mất cấu trúc, chỉ tìm một lần và không kiểm tra chất lượng kết quả.

2.2. RAPTOR

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) xây cây tóm tắt: các đoạn 100 token được gom cụm bằng Gaussian Mixture Model sau khi giảm chiều bằng UMAP, mỗi cụm được LLM tóm tắt thành nút cha, lặp lại cho đến khi đủ số tầng. Truy vấn tìm trên toàn bộ các tầng (collapsed tree). Cách này giúp trả lời câu hỏi cần cái nhìn tổng thể, nhưng cụm là cụm theo ngữ nghĩa, không theo cấu trúc tài liệu, nên nút không gắn được với vị trí dòng trong văn bản gốc.

2.3. Corrective RAG (CRAG)

CRAG bổ sung bước đánh giá sau truy hồi: một bộ chấm điểm (grader) loại các đoạn không liên quan, và nếu ngữ cảnh chưa đủ thì hệ thống truy hồi lại với truy vấn khác. Trong dự án, ý tưởng này được mở rộng thành agent có vòng lặp và nhiều công cụ.

2.4. Agentic RAG và LangGraph

Agentic RAG giao cho LLM quyền chọn công cụ và tham số tìm kiếm ở mỗi vòng. Dự án dùng LangGraph, thư viện mô hình hoá luồng agent thành đồ thị trạng thái (StateGraph). Cơ chế `Send` cho phép rẽ nhánh động và chạy song song nhiều đồ thị con, được dùng để xử lý các khía cạnh của câu hỏi cùng lúc.

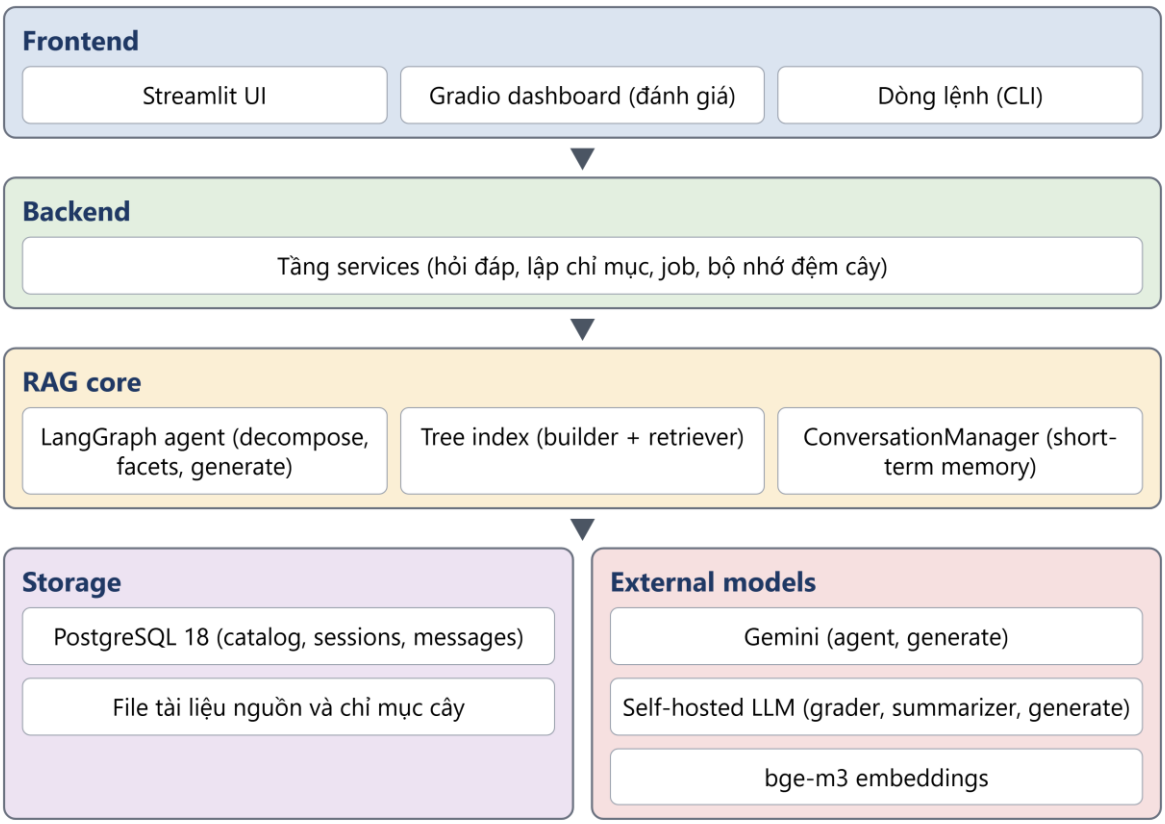
2.5. LLM-as-judge

Chất lượng câu trả lời được chấm bằng một LLM so với đáp án tham chiếu theo bốn tiêu chí độc lập (Accuracy, Completeness, Relevance, Faithfulness) trên thang 0–10. Mỗi tiêu chí dùng một lời nhắc riêng, chạy song song để tránh hiệu ứng "halo" (điểm tiêu chí này kéo theo tiêu chí khác).

CHƯƠNG 3. KIẾN TRÚC TỔNG THỂ

3.1. Các thành phần

Hệ thống chia thành bốn lớp. Lớp giao diện gồm Streamlit và dashboard Gradio. Lớp backend là tầng dịch vụ dùng chung (hỏi đáp, lập chỉ mục, hàng đợi job, bộ nhớ đệm cây). Lớp lõi RAG gồm agent LangGraph, chỉ mục cây và ConversationManager. Lớp lưu trữ gồm PostgreSQL (danh mục, phiên chat, tin nhắn) và các file trên đĩa (Markdown nguồn, chỉ mục dạng pickle).



Hình 1. Kiến trúc tổng thể của hệ thống

Điểm thiết kế đáng chú ý là tầng dịch vụ không phụ thuộc Streamlit nên giao diện, dòng lệnh và dashboard cùng gọi chung một mã nguồn nghiệp vụ. Streamlit gọi trực tiếp services trong cùng tiến trình, không đi qua HTTP. Repo có kèm một lớp REST API (FastAPI) mỏng bọc các services này nhưng giao diện không dùng đến nên báo cáo không đi sâu.

3.2. Cấu trúc mã nguồn

Cấu trúc thư mục chính của dự án:

```
slidegen-rag/
├── src/slidegen_rag/
│   ├── agent/        # đồ thị agent, node, công cụ tìm kiếm
│   ├── tree/         # xây chỉ mục cây và truy hồi
│   ├── evaluate/     # chạy đánh giá, judge, dashboard Gradio
│   ├── services/     # tầng backend dùng chung
│   ├── memory/       # mô hình dữ liệu, repository, bộ nhớ hội thoại
│   └── config.py     # cấu hình
├── ui/               # giao diện Streamlit
├── migrations/       # Alembic, cấu trúc cơ sở dữ liệu
├── tests/            # kiểm thử
├── data/             # tài liệu nguồn, chỉ mục, kết quả đánh giá
├── docs/             # tài liệu, báo cáo
└── pyproject.toml    # khai báo phụ thuộc
```

Bảng 2. Các phần chính của dự án

Phần	Nội dung
Agent	Đồ thị agent, các node, công cụ tìm kiếm, theo dõi quỹ đạo
Cây chỉ mục	Xây chỉ mục cây và truy hồi trên cây
Đánh giá	Chạy đánh giá, judge, tính chỉ số, so sánh các lần chạy, dashboard Gradio
Dịch vụ (backend)	Hỏi đáp, lập chỉ mục, hàng đợi job, bộ nhớ đệm, đọc tài liệu, trích dẫn
Bộ nhớ và cơ sở dữ liệu	Mô hình dữ liệu, repository, migration
Giao diện	Ứng dụng Streamlit

3.3. Luồng một lượt hỏi đáp

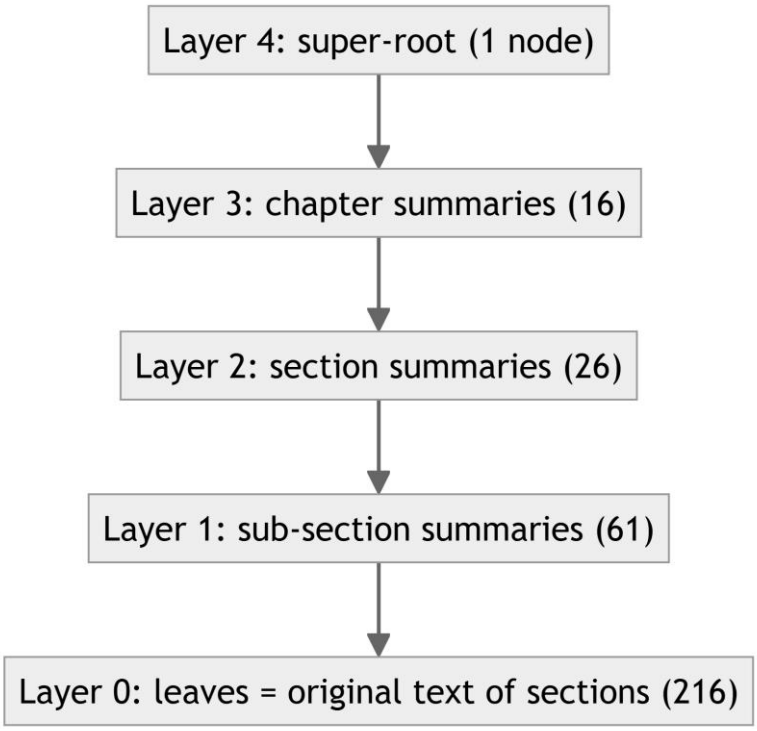
1. Người dùng chọn tài liệu trong phạm vi (scope) của phiên chat và gửi câu hỏi.
2. Dịch vụ hỏi đáp nạp các cây chỉ mục của tài liệu trong scope và gộp thành một cây tìm kiếm (nếu hơn 3 tài liệu thì lọc trước còn 3).
3. ConversationManager viết lại câu hỏi nối tiếp thành câu hỏi độc lập dựa trên lịch sử.
4. Agent phân rã câu hỏi, truy hồi có sửa lỗi theo từng khía cạnh rồi sinh câu trả lời.
5. Trích dẫn được trích ra, lượt hỏi được ghi vào PostgreSQL và lịch sử được nén nếu cần.

CHƯƠNG 4. LẬP CHỈ MỤC: CÂY HEADING PHÂN CẤP

Phần do Đinh Tuấn Quang phụ trách.

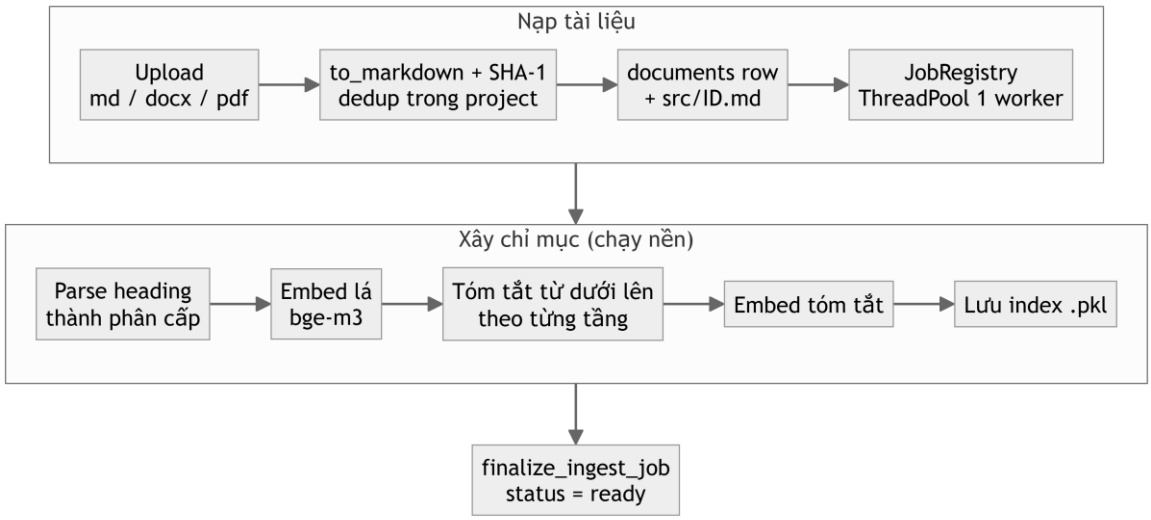
4.1. Ý tưởng

Thay vì cắt văn bản theo độ dài hay gom cụm, dự án dùng chính cấu trúc heading của tài liệu Markdown làm cấu trúc cây. Nút lá là nguyên văn các mục (không cắt, không viết lại, chỉ được nhúng vector). Nút cha là bản tóm tắt do LLM sinh từ các nút con, tính từ dưới lên theo từng tầng. Mỗi nút giữ lại đường dẫn heading và số dòng đầu và dòng cuối trong tài liệu nguồn, nhờ đó có thể trích dẫn chính xác và các công cụ "đi xuống / đi sang bên" trong cây hoạt động được.



Hình 2. Cấu trúc cây chỉ mục của giáo trình (320 nút, 5 tầng)

4.2. Quy trình xây cây



Hình 3. Quy trình nạp và lập chỉ mục tài liệu

- **Phân tích heading**: độ sâu lấy theo đánh số (ví dụ 1.1.2.1), từ khoá chương ép về cấp 1, loại bỏ nhiễu dấu đầu dòng do OCR; nếu có nhiều gốc thì thêm một siêu gốc.
- **Nhúng vector**: mô hình BAAI/bge-m3 (1024 chiều). Ưu tiên gọi dịch vụ embedding từ xa, nếu lỗi thì dùng sentence-transformers cục bộ.
- **Tóm tắt từ dưới lên**: gom theo từng tầng và chạy theo lô; độ dài tóm tắt tối đa 2000 token. Mô hình tóm tắt mặc định là deepseek-v4-flash, dự phòng gemini-2.5-flash.
- **Lưu trữ**: toàn bộ cây (văn bản, vector, quan hệ cha–con) được lưu thành file pickle; không dùng cơ sở dữ liệu vector, tìm kiếm là cosine brute-force trên ma trận đã chuẩn hoá.

4.3. Chi phí lập chỉ mục

Bảng 3. Chi phí xây chỉ mục cho giáo trình Pháp luật đại cương

Chỉ số	Giá trị
Kích thước tài liệu	534 nghìn ký tự, 5.491 dòng
Số nút / số tầng	320 nút / 5 tầng (216 lá, 61, 26, 16, 1)
Thời gian phân tích heading	7 ms
Số lần gọi LLM tóm tắt	103
Số vector nhúng	320
Tổng thời gian	khoảng 5 phút trở lên

4.4. So sánh với RAPTOR

Dự án ban đầu dùng RAPTOR (chunk 100 token, GMM + UMAP, tối đa 5 tầng) nhưng đã thay bằng cây heading vì hàm `split_text` của RAPTOR làm mất vị trí dòng, khiến các công cụ truy hồi theo cấu trúc không thể hoạt động. Kết quả đánh giá ở Chương 6 cũng cho thấy cây heading cho điểm cao hơn rõ rệt: 7,89 so với 6,93 khi dùng cùng quy trình CRAG.

Bảng 4. So sánh các kiểu chỉ mục

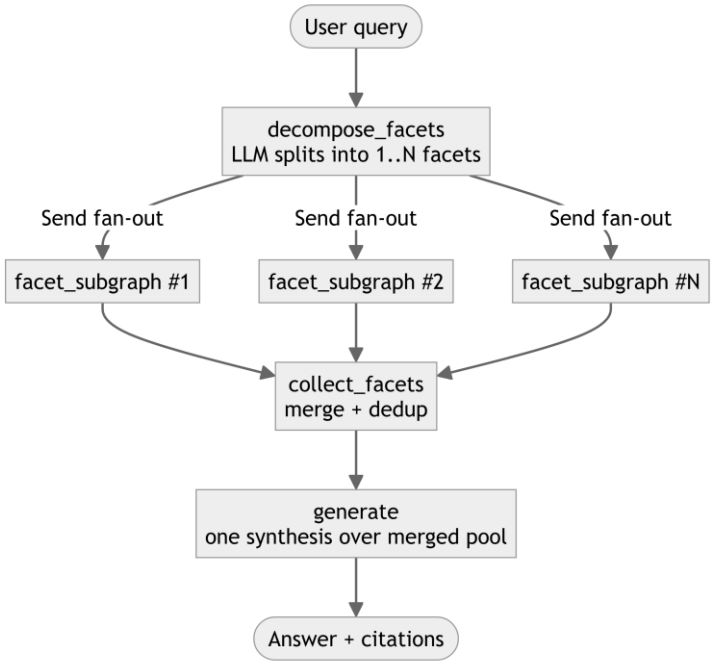
Tiêu chí	**Naive RAG**	**RAPTOR**	**Cây heading (dự án)**
Đơn vị lưu	Đoạn cố định	Đoạn 100 token + tóm tắt theo cụm	Mục nguyên văn + tóm tắt theo cấu trúc
Cấu trúc	Phẳng	Cây theo cụm ngữ nghĩa	Cây theo heading
Truy ngược dòng nguồn	Hạn chế	Không	Có (số dòng đầu/cuối)
Đi lên/xuống/sang bên	Không	Không	Có (expand_node, find_in_subtree)
Chi phí lập chỉ mục	Thấp	Cao (41 MB pickle)	Trung bình (4,5 MB pickle)

CHƯƠNG 5. AGENT TRUY HỒI

Phần do Đinh Tuấn Quang phụ trách.

5.1. Đồ thị cha: phân rã và gộp khía cạnh

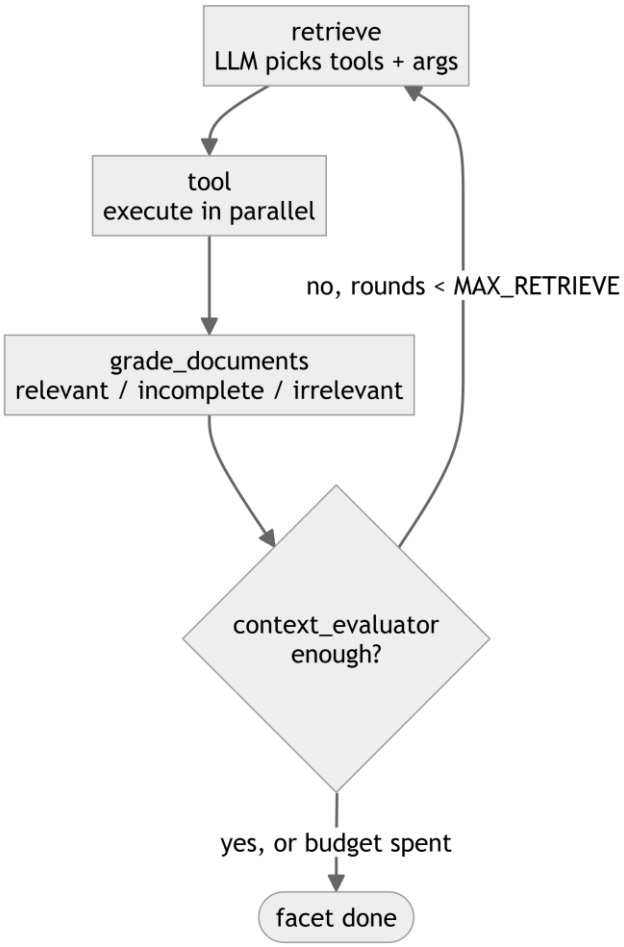
Đồ thị cha có bốn bước: `decompose_facets` → fan-out `Send` → `facet_subgraph` (chạy song song N bản) → `collect_facets` → `generate`. Node phân rã chia câu hỏi thành tối đa 4 khía cạnh. Nếu câu hỏi chứa dấu hiệu so sánh ("so sánh", "phân biệt"…) mà chỉ ra một khía cạnh thì hệ thống hỏi LLM thêm một lần để tách; khi lỗi thì lùi về một khía cạnh duy nhất.



Hình 4. Đồ thị cha của agent

5.2. Đồ thị con: vòng truy hồi có sửa lỗi

Mỗi khía cạnh chạy một vòng lặp riêng với ngân sách riêng: `retrieve` (LLM chọn công cụ và tham số) → `tool` (thực thi song song) → `grade_documents` (lọc theo mức liên quan) → `context_evaluator` (đủ chưa?). Vòng lặp kết thúc khi ngữ cảnh đủ, khi hết `MAX_RETRIEVE = 5` vòng, hoặc khi từ vòng 3 trở đi hai vòng liên tiếp không thu thêm đoạn mới nào (no-progress guard).



Hình 5. Vòng truy hồi có sửa lỗi của mỗi khía cạnh

Bảng 5. Vai trò của các node

Node	Quyết định	Ghi vào state
`decompose_facets`	Câu hỏi có nhiều khía cạnh không, mỗi câu hỏi con là gì	facet_queries
`retrieve`	Gọi công cụ nào, tham số gì; loại cặp (tool, query) đã thử	tool_calls
`tool`	Thực thi song song, tách thành đoạn, khử trùng giữa các vòng	pending_documents, search_history, node_digest
`grade_documents`	Đoạn nào giữ lại; đoạn nào đúng chủ đề nhưng chưa đủ (cần drill-down)	documents, document_priorities, drill_candidates
`context_evaluator`	Ngữ cảnh đã đủ chưa, nếu chưa thì cần tìm gì	context_sufficient, missing_aspects, feedback
`collect_facets`	Gộp và khử trùng tài liệu giữa các khía cạnh	documents, facet_sufficiency
`generate`	Sinh câu trả lời trong giới hạn 32.000 ký tự ngữ cảnh	generation

5.3. Các công cụ tìm kiếm

Bảng 6. Công cụ của agent

Công cụ	Chức năng
`semantic_search`	Cosine trên mọi tầng của cây (collapsed tree), top_k = 3, tối đa 4000 token
`keyword_search`	BM25 (rank_bm25) trên mọi nút
`expand_node`	Đi tới nút con, nút cha hoặc nút anh em của một nút
`find_in_subtree`	Tìm BM25 hoặc ngữ nghĩa trong phạm vi một nhánh
`web_search`	Tìm web bằng Tavily (chỉ bật khi có khoá API; tắt khi đánh giá)

LLM của node `retrieve` được ép gọi ít nhất một công cụ mỗi vòng (`tool_choice="any"`). Mỗi bước giới hạn 6 lệnh gọi công cụ, sau khi từng ghi nhận một lần chạy sinh tới 517 lệnh gọi trong một bước.

5.4. Chấm điểm đoạn và đánh giá ngữ cảnh

- **Grader** chạy bất đồng bộ, 16 lệnh gọi đồng thời, ba nhãn: `relevant` (ưu tiên 2), `relevant_incomplete` (1), `irrelevant` (0). Nếu tất cả bị loại, hệ thống vẫn giữ 5 đoạn đầu (hard floor).
- Các đoạn `relevant_incomplete` trở thành **ứng viên drill-down** (cần chi tiết, cần anh em, cần ngữ cảnh), tối đa 10, giúp vòng sau đi xuống đúng nhánh.
- **Context evaluator** trả về SUFFICIENT/MISSING/FEEDBACK; phản hồi này cùng lịch sử tìm kiếm là đầu vào để LLM viết truy vấn mới (không có node viết lại truy vấn riêng).
- **Ngân sách ngữ cảnh**: 32.000 ký tự, lấp theo mức ưu tiên rồi đến độ mới.

5.5. Mô hình ngôn ngữ và giám sát

Bảng 7. Mô hình sử dụng

Vai trò	Mô hình
Retrieve, decompose, context_evaluator, contextualize	Gemini (gemini-3.5-flash-lite)
Grader (max_tokens = 24)	Mô hình tự host qua endpoint tương thích OpenAI
Generate	Gemini hoặc mô hình tự host (chuyển đổi được, tự lùi về Gemini khi lỗi)
Tóm tắt khi lập chỉ mục	deepseek-v4-flash, dự phòng gemini-2.5-flash
Embedding	BAAI/bge-m3

Dự án ghi lại quỹ đạo từng node (thời gian, công cụ gọi, số vòng), đồng thời hỗ trợ LangSmith để theo dõi lời gọi LLM. `TokenTracker` tính số token và chi phí.

CHƯƠNG 6. ĐÁNH GIÁ

Phần do Đinh Tuấn Quang phụ trách.

6.1. Phương pháp

Bộ dữ liệu gồm 30 câu hỏi pháp luật tiếng Việt và đáp án tham chiếu. Mỗi câu trả lời được chấm bằng bốn lời nhắc judge độc lập, mỗi tiêu chí thang 0–10; điểm tổng là trung bình bốn tiêu chí. Judge chạy ở nhiệt độ 0, ưu tiên mô hình tự host, dự phòng Gemini.

Bảng 8. Các chỉ số đánh giá

Chỉ số	Ý nghĩa
Accuracy	Đúng so với đáp án mẫu
Completeness	Bao quát các ý cốt lõi
Relevance	Bám sát câu hỏi, không lan man
Faithfulness	Chỉ dựa vào ngữ cảnh đã truy hồi, không bịa
Chunk coverage	% từ khoá của đáp án mẫu xuất hiện trong các đoạn truy hồi
Answer coverage	% từ khoá của đáp án mẫu xuất hiện trong câu trả lời
Utilization	Answer coverage chia cho chunk coverage (strict)

Độ nhiễu của judge khoảng ±0,06 (chấm lại cùng một bộ câu trả lời bốn lần cho 8,03 / 8,03 / 8,05 / 8,09), vì vậy chênh lệch dưới khoảng 0,1 được coi là nhiễu.

6.2. Các hệ thống so sánh

Bảng 9. Các hệ thống được so sánh

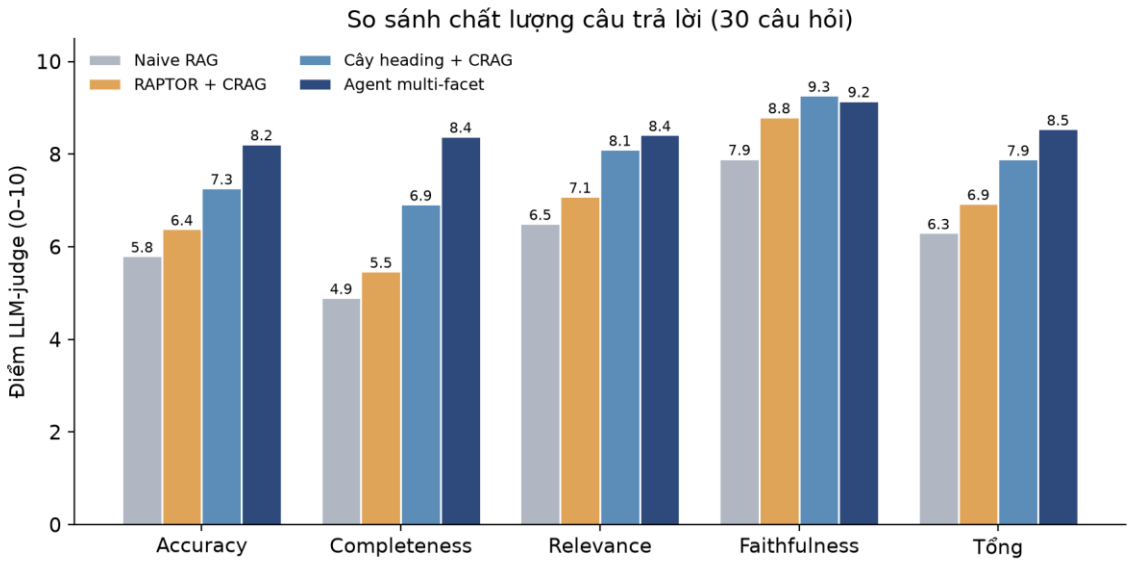
Hệ thống	Chỉ mục	Truy hồi / sinh	Nguồn số liệu
Naive RAG	Đoạn cố định, nhúng bge-m3	Top-k một lần rồi sinh	Nhóm tự chạy trên cùng 30 câu hỏi và cùng judge
RAPTOR + CRAG	GMM + UMAP, đoạn 100 token	CRAG 5 node (gemini-2.5-flash)	Lần chạy 14/09/2026
Cây heading + CRAG	Cây heading v1	CRAG 5 node	Lần chạy 14/09/2026
Agent multi-facet (dự án)	Cây heading v2	Phân rã + vòng truy hồi song song	Lần chạy 15/09/2026

6.3. Kết quả

Bảng 10. Điểm trung bình trên 30 câu hỏi

Hệ thống	Acc	Comp	Rel	Faith	Tổng
Naive RAG	5,8	4,9	6,5	7,9	6,3
RAPTOR + CRAG	6,38	5,47	7,08	8,80	6,93
Cây heading + CRAG	7,27	6,92	8,10	9,27	7,89
Agent multi-facet	8,22	8,38	8,42	9,15	8,54

Số liệu Naive RAG do nhóm tự chạy trên cùng bộ 30 câu hỏi và cùng judge; các dòng còn lại lấy từ kết quả các lần chạy đã lưu của dự án.



Hình 6. So sánh điểm LLM-judge giữa các hệ thống

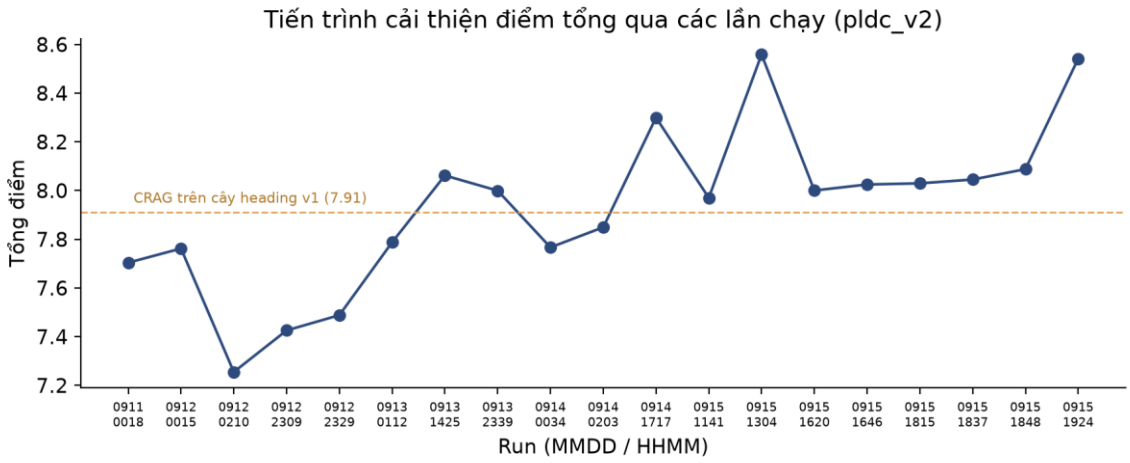
Cây heading kết hợp CRAG đã tăng khoảng 0,96 điểm so với RAPTOR, và agent multi-facet tiếp tục tăng thêm khoảng 0,65 điểm, lớn hơn nhiều mức nhiễu ±0,06. Cải thiện lớn nhất nằm ở Completeness (5,47 → 8,38): việc phân rã khía cạnh và drill-down giúp thu đủ các ý của câu hỏi tổng hợp. Faithfulness ở mọi hệ thống đều cao (8,8–9,3) vì câu trả lời được sinh từ ngữ cảnh truy hồi.

Bảng 11. Chỉ số chẩn đoán của lần chạy agent multi-facet

Chỉ số	Giá trị
Chunk coverage	81,1% (strict 79,0%)
Answer coverage	73,5%
Utilization	94,0%
Số đoạn truy hồi trung bình / câu	10,0
Câu hoàn thành	30/30
Chi phí và thời gian toàn bộ run	khoảng 0,27 USD, 31,5 phút

6.4. Tiến trình cải thiện

Toàn bộ các lần chạy đều được giữ lại trong thư mục kết quả, kể cả những lần điểm giảm. Điểm tổng tăng từ khoảng 7,7 (CRAG vòng lặp đơn) lên 8,06 khi thêm phân rã đa khía cạnh, và đạt đỉnh 8,56 khi sinh câu trả lời bằng Gemini.



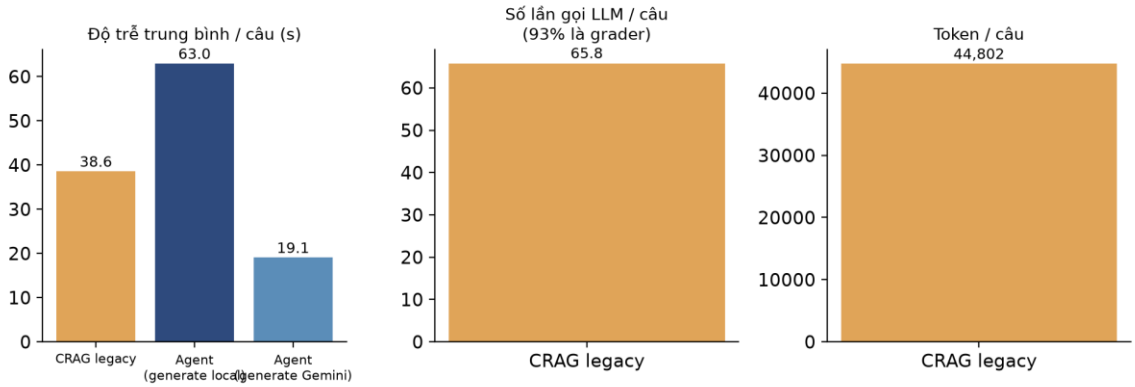
Hình 7. Điểm tổng qua các lần chạy

Phân tích theo số khía cạnh cho thấy các câu 2 khía cạnh có điểm trung bình 8,49 (n = 13), cao hơn câu 1 khía cạnh là 8,05 (n = 16).

6.5. Chi phí và độ trễ

Bảng 12. Độ trễ theo nhà cung cấp sinh câu trả lời

	Run 8,54 (sinh bằng mô hình tự host)	Run 8,56 (sinh bằng Gemini)
Trung bình / câu	63,0 s	19,1 s
Trung vị	62,6 s	20,1 s
Nhanh nhất / chậm nhất	22,5 s / 100,8 s	5,6 s / 56,0 s
Tổng 30 câu	31,5 phút	9,6 phút



Hình 8. Độ trễ và chi phí so với CRAG legacy

Vòng truy hồi có sửa lỗi rất rẻ: toàn bộ khoảng 200 vòng retrieve + tool + grade + evaluate chỉ tốn khoảng 14 giây cho mỗi câu. Thời gian chủ yếu nằm ở bước sinh câu trả lời (76% ở run dùng mô hình tự host). Đổi nhà cung cấp sinh làm độ trễ giảm 3 lần nhưng điểm chỉ lệch 0,02 (trong ngưỡng nhiễu), nghĩa là độ trễ và chất lượng là hai núm điều chỉnh độc lập. Trung bình agent dùng 3,9/5 vòng, 20/30 câu dùng hết ngân sách vòng.

Để so sánh, baseline CRAG legacy dùng trung bình 44.802 token và 65,8 lệnh gọi LLM mỗi câu (93% là lệnh gọi của grader), và 18/30 câu (60%) hết số vòng truy hồi mà vẫn chưa đủ ngữ cảnh.

CHƯƠNG 7. BACKEND

Phần do Nguyễn Duy Long phụ trách.

Backend là một tầng dịch vụ (services) viết thuần Python, tách khỏi giao diện để Streamlit, dòng lệnh và dashboard đánh giá cùng dùng chung. Tầng này gồm các thành phần sau:

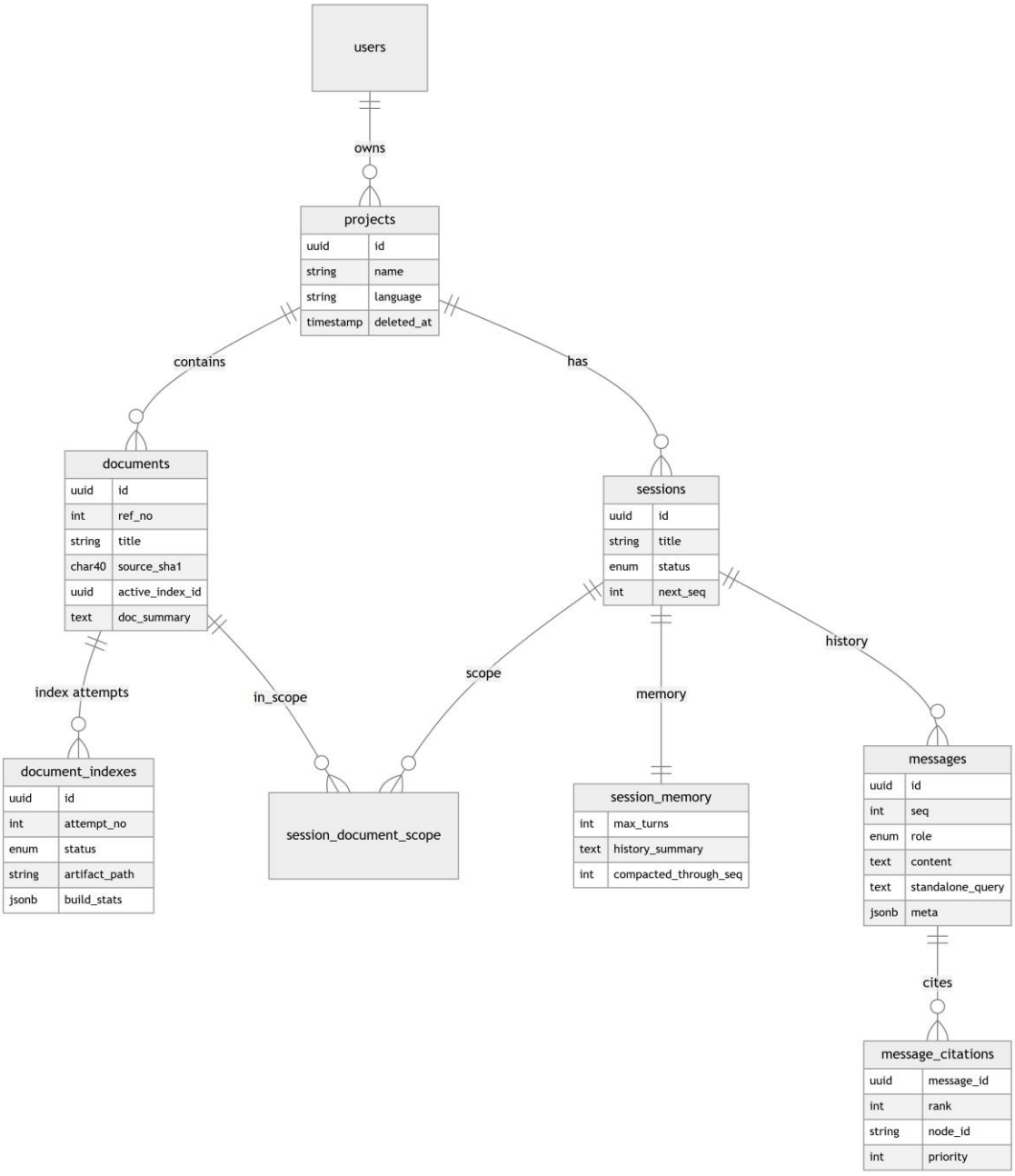
- **Dịch vụ hỏi đáp**: thực hiện trọn một lượt chat, từ nạp lịch sử, viết lại câu hỏi, chạy agent đến lưu câu trả lời và trích dẫn.
- **Dịch vụ lập chỉ mục và hàng đợi job**: chạy việc lập chỉ mục ở luồng nền bằng `ThreadPoolExecutor`, có theo dõi tiến độ và cho phép huỷ.
- **Bộ nhớ đệm cây và lọc tài liệu**: giữ cây chỉ mục đã nạp để dùng lại; khi có nhiều tài liệu thì chọn trước vài tài liệu liên quan nhất.
- **Đọc tài liệu**: chuyển các định dạng md, txt, docx và pdf về Markdown trước khi lập chỉ mục.
- **Xử lý lỗi**: dùng các ngoại lệ riêng của dự án để giao diện báo lỗi rõ ràng, ví dụ chưa có tài liệu sẵn sàng hay mất kết nối cơ sở dữ liệu.

Hạn chế hiện tại: chưa có đăng nhập nhiều người dùng, câu trả lời chưa được truyền theo dòng (streaming) và hàng đợi job chỉ nằm trong bộ nhớ.

CHƯƠNG 8. CƠ SỞ DỮ LIỆU

Phần do Nguyễn Duy Long phụ trách.

Dữ liệu quan hệ lưu trong PostgreSQL 18, truy cập bất đồng bộ bằng SQLAlchemy 2 và asyncpg; cấu trúc bảng được quản lý bằng Alembic. Có 9 bảng, quan hệ giữa chúng được mô tả ở Hình 9.



Hình 9. Sơ đồ quan hệ thực thể (ERD)

Bảng 13. Các nhóm bảng chính

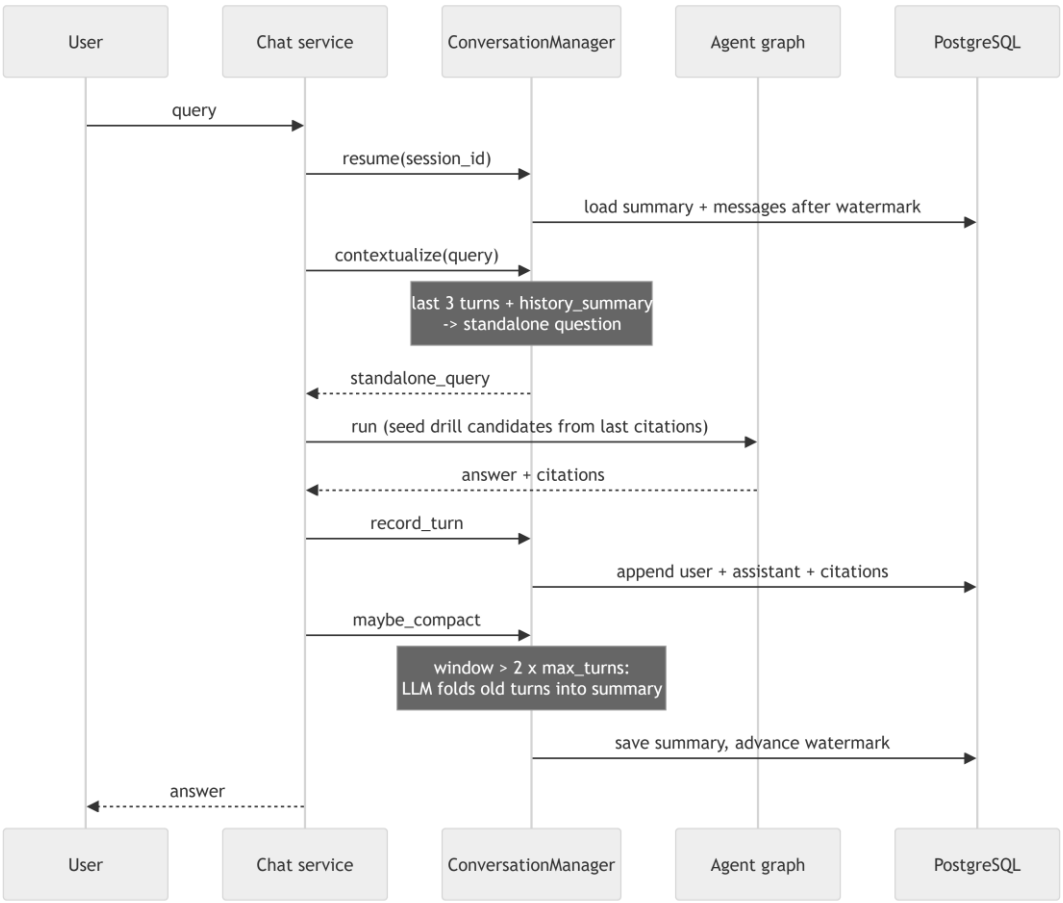
Nhóm	Nội dung
Người dùng và project	Chủ sở hữu và không gian làm việc, có xoá mềm
Tài liệu và chỉ mục	Thông tin tài liệu và từng lần lập chỉ mục
Phiên chat	Phiên hội thoại và phạm vi tài liệu được dùng
Bộ nhớ và tin nhắn	Tin nhắn, trích dẫn và tóm tắt lịch sử

File Markdown nguồn và chỉ mục cây (dạng pickle) được lưu trên đĩa theo từng project; vector không dùng cơ sở dữ liệu vector riêng. Truy cập dữ liệu đi qua các interface (`typing.Protocol`) có hai bản cài đặt: PostgreSQL và trong bộ nhớ, nhờ đó chạy được dòng lệnh khi không có cơ sở dữ liệu.

CHƯƠNG 9. BỘ NHỚ HỘI THOẠI

Phần do Nguyễn Duy Long phụ trách.

Bộ nhớ ngắn hạn do lớp `ConversationManager` đảm nhiệm, lịch sử được lưu trong PostgreSQL và nạp lại trước mỗi lượt. Các cơ chế chính:



Hình 10. Luồng bộ nhớ trong một lượt hỏi đáp

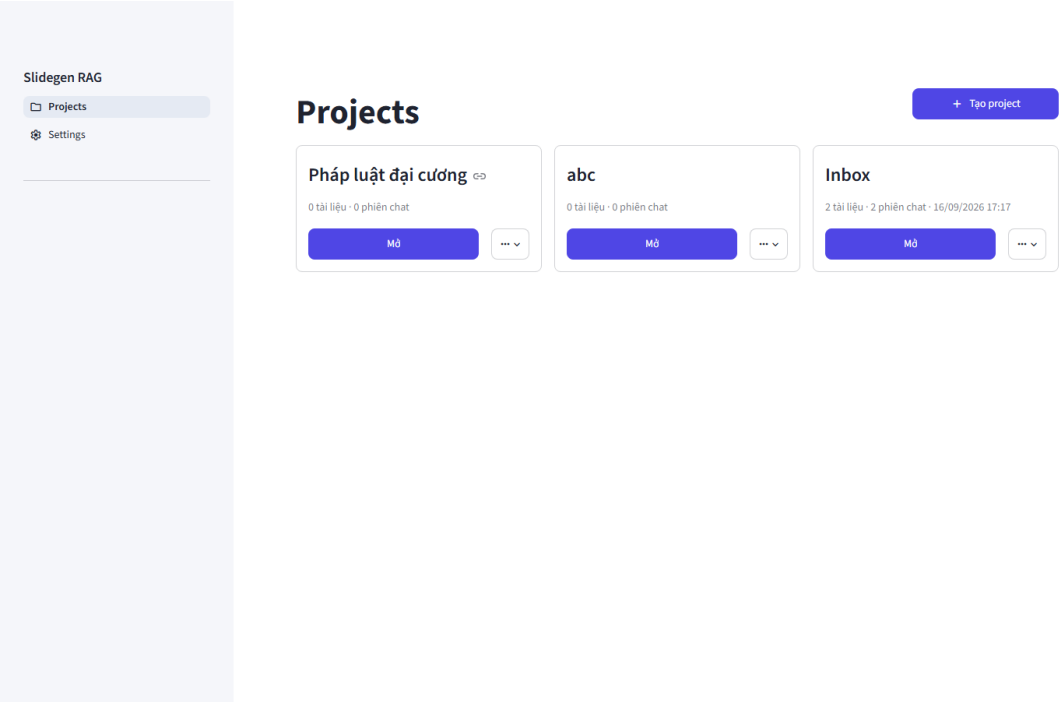
- **Cửa sổ trượt**: giữ 5 lượt hỏi đáp gần nhất.
- **Nén lịch sử**: khi hội thoại dài, các tin cũ được LLM tóm tắt lại để không phình ngữ cảnh.
- **Viết lại câu hỏi**: câu hỏi nối tiếp (“Vậy còn…?”) được viết lại thành câu hỏi đầy đủ trước khi đưa vào agent.
- **Gợi ý truy hồi**: các đoạn đã trích dẫn ở câu trước giúp câu sau tìm đúng vùng tài liệu nhanh hơn.
- **Chế độ dự phòng**: nếu cơ sở dữ liệu lỗi, hội thoại tiếp tục trong bộ nhớ thay vì dừng hẳn.

CHƯƠNG 10. GIAO DIỆN STREAMLIT

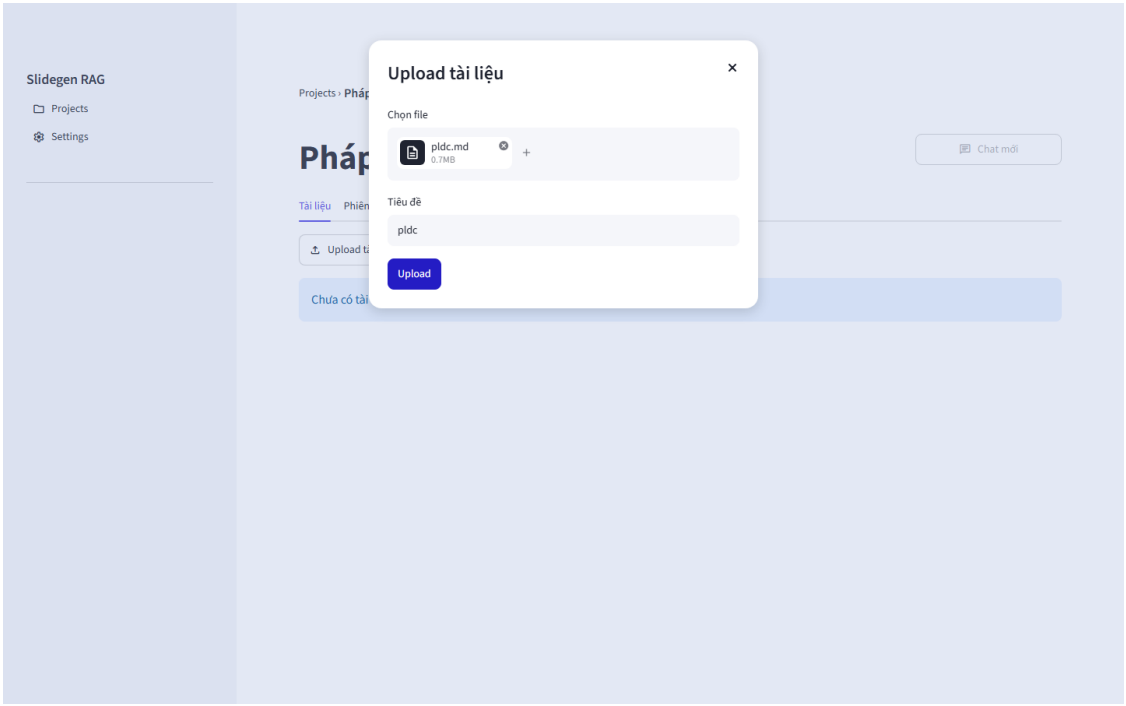
Phần do Lê Anh Tú phụ trách.

10.1. Các chức năng chính

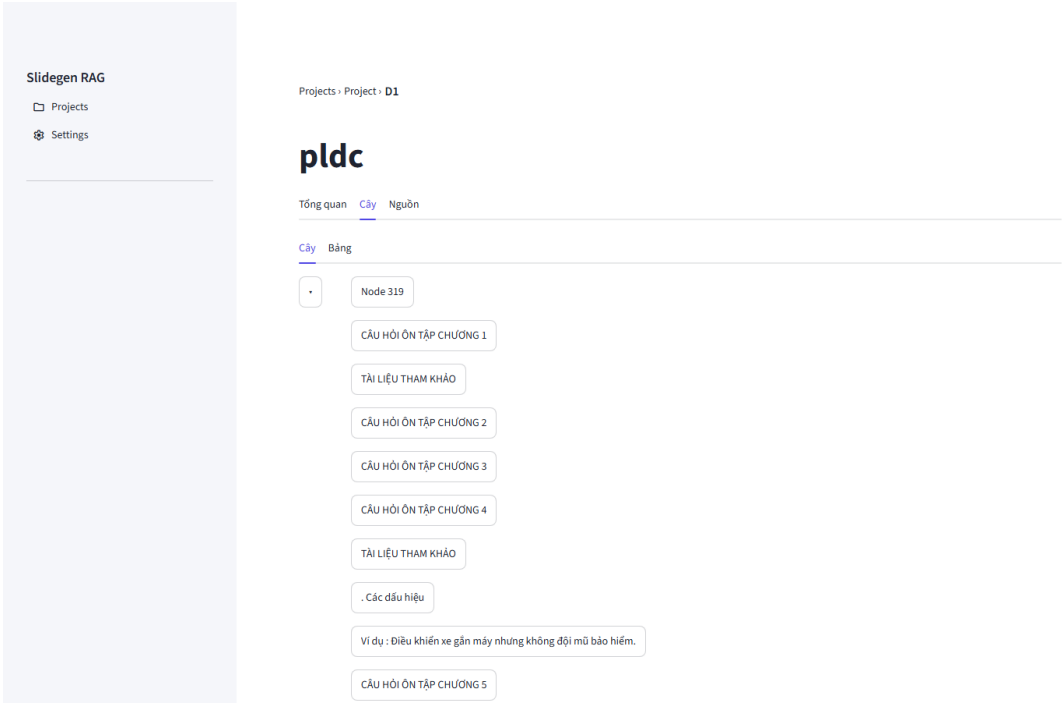
Giao diện xây bằng Streamlit, gọi trực tiếp tầng services trong cùng tiến trình. Các chức năng chính gồm quản lý project, upload tài liệu và theo dõi tiến độ lập chỉ mục, xem cây chỉ mục, chat có trích dẫn và trang cài đặt. Tiến trình của agent được hiển thị theo từng bước ngay trong lúc chạy.



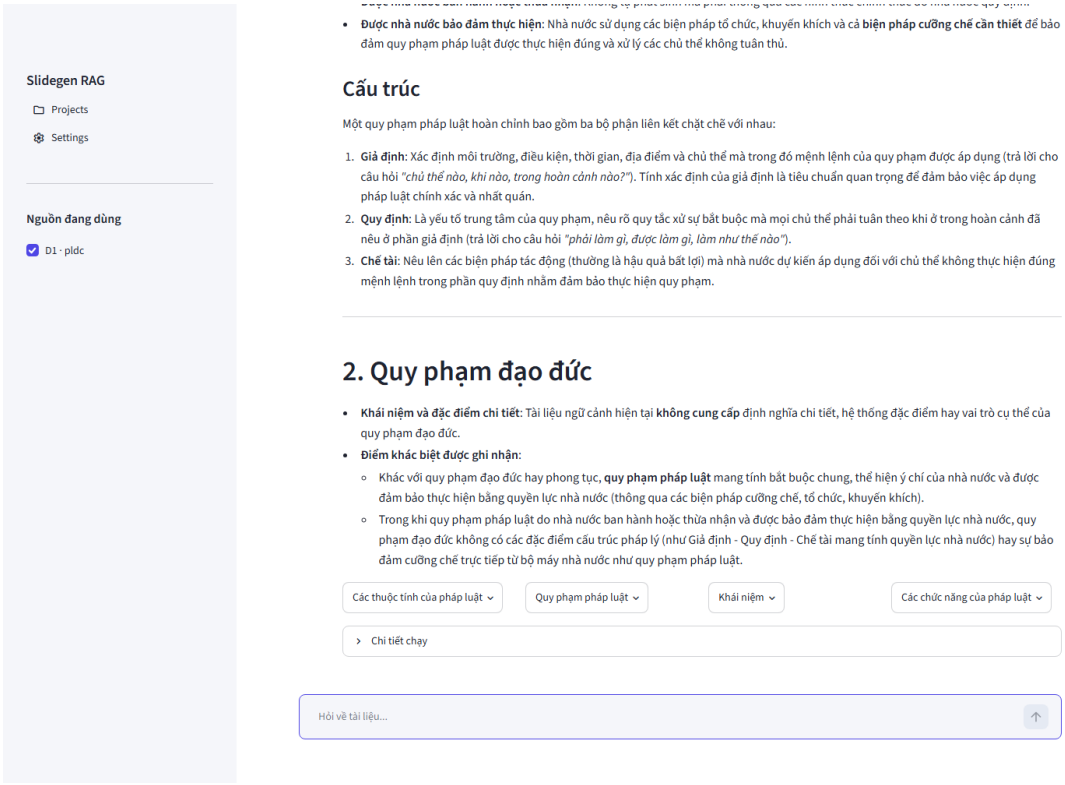
Hình 11. Trang danh sách project



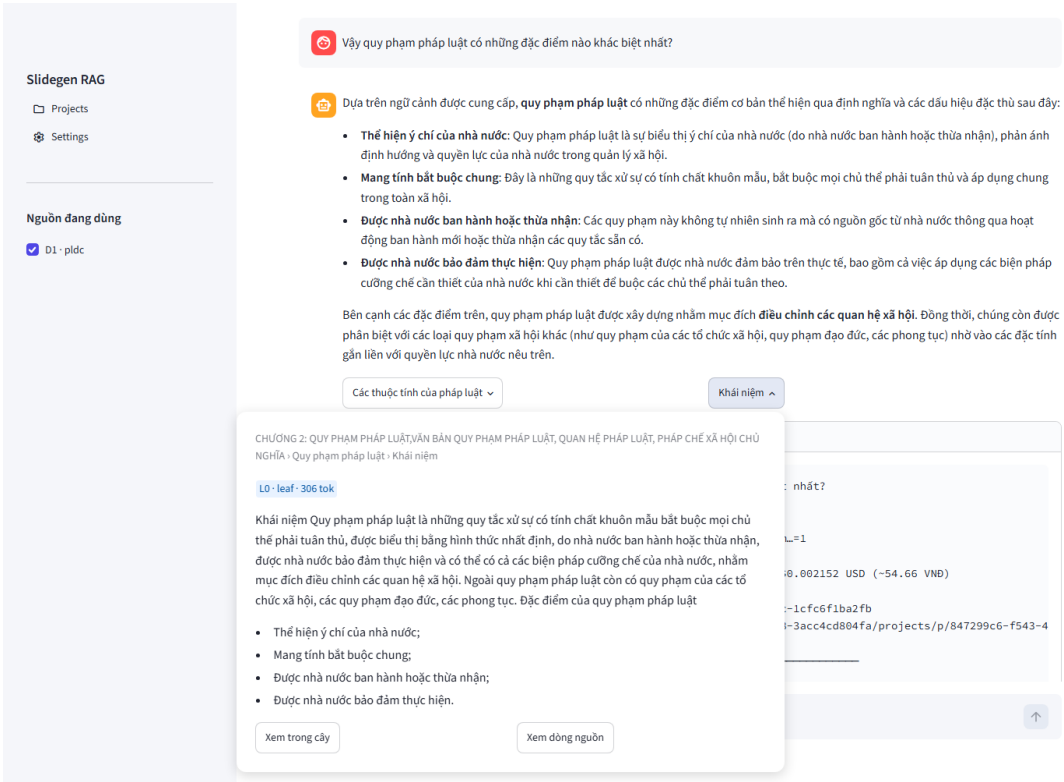
Hình 12. Tài liệu đang được lập chỉ mục



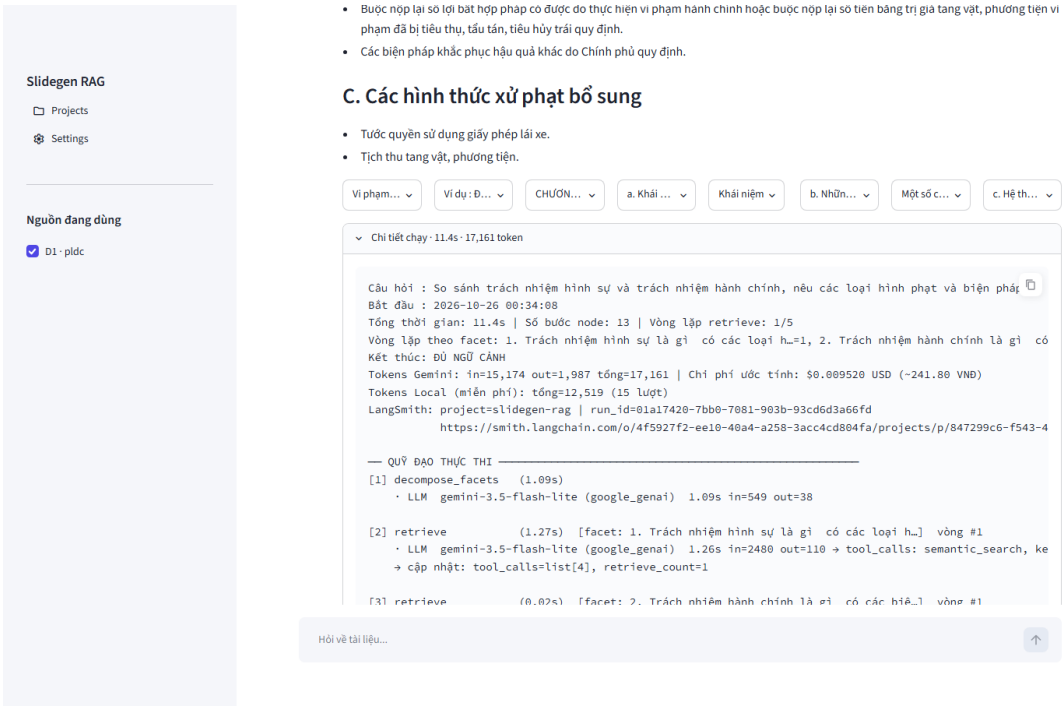
Hình 13. Xem cây chỉ mục của tài liệu



Hình 14. Câu trả lời kèm các chip trích dẫn



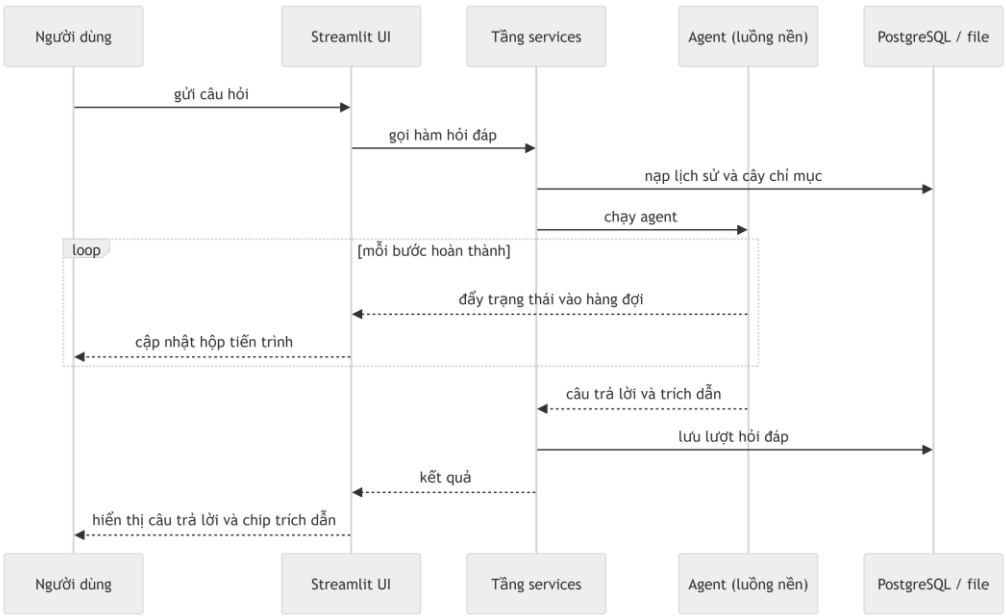
Hình 15. Popover của một trích dẫn



Hình 16. Chi tiết lượt chạy: thời gian, số vòng, token, chi phí và quỹ đạo

10.2. Cách giao diện tương tác với các phần khác

Giao diện không chạy logic nghiệp vụ mà chỉ gọi sang các phần khác của dự án. Hình 16 mô tả một lượt hỏi đáp và bảng bên dưới tóm tắt các điểm giao tiếp.



Hình 17. Giao diện tương tác với tầng services, agent và cơ sở dữ liệu trong một lượt chat

Bảng 14. Các điểm giao tiếp của giao diện

Thành phần	Cách giao diện tương tác
Tầng services (backend)	Gọi trực tiếp các hàm trong cùng tiến trình, không đi qua HTTP
Agent	Chạy ở luồng nền vì agent là bất đồng bộ còn Streamlit chạy đồng bộ; mỗi bước hoàn thành được đẩy vào hàng đợi để giao diện hiển thị dần
Lập chỉ mục	Giao diện tạo job nền, một đoạn giao diện tự làm mới mỗi giây để cập nhật thanh tiến độ và nút huỷ
PostgreSQL và bộ nhớ	Danh sách project, tài liệu, phiên chat và lịch sử được đọc, ghi qua tầng services; mở lại phiên là nạp lại hội thoại
Cây chỉ mục	Trích dẫn trong câu trả lời liên kết tới nút trong cây và dòng trong tài liệu nguồn
Trạng thái trang	Mã project, tài liệu và phiên được lưu trên địa chỉ trang nên tải lại trang không mất vị trí

Ngoài Streamlit, dự án có dashboard Gradio để chạy và so sánh các lần đánh giá chất lượng (chương 6).

CHƯƠNG 11. CÔNG NGHỆ VÀ THƯ VIỆN

Dự án viết bằng Python 3.12, quản lý môi trường và phụ thuộc bằng uv. Bảng dưới liệt kê các thư viện chính cùng phiên bản đang dùng.

Bảng 15. Thư viện chính

Nhóm	Thư viện (phiên bản)	Vai trò
Cơ sở dữ liệu	sqlalchemy 2.0.52, asyncpg 0.31.0, alembic 1.20.0	ORM bất đồng bộ, driver, migration
Agent	langgraph 1.2.10, langchain 1.3.14, langchain-google-genai 4.3.4, langchain-openai 1.4.3, langsmith 0.10.12	Đồ thị agent, gọi LLM, theo dõi
Truy hồi	sentence-transformers 5.6.1, torch 2.13.0, rank-bm25 0.2.2, tiktoken, numpy, scipy, tavily	Embedding bge-m3, BM25, cosine, web search
Giao diện	streamlit 1.64.0, gradio 6.21.0, matplotlib	UI chính, dashboard đánh giá, biểu đồ
Tài liệu	pypdf 6.21.2, python-docx 1.2.0	Đọc pdf, docx
Kiểm thử	pytest 9.1.1, pytest-asyncio 1.4.0, httpx 0.28.1, ruff 0.16.6	Test, kiểm tra mã, định dạng

CHƯƠNG 12. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

12.1. Kết quả đạt được

- Chỉ mục cây heading giữ cấu trúc tài liệu và cho phép trích dẫn đến từng dòng nguồn.
- Agent đa khía cạnh với vòng truy hồi có sửa lỗi đạt 8,54/10 trên 30 câu hỏi, cao hơn CRAG trên cây heading (7,89) và CRAG trên RAPTOR (6,93).
- Hệ thống hoàn chỉnh: tầng services, PostgreSQL với migration, bộ nhớ hội thoại có nén lịch sử, giao diện Streamlit và dashboard đánh giá.

12.2. Hạn chế

- Thử nghiệm mới trên một tài liệu tiếng Việt và 30 câu hỏi.
- Chat chưa streaming; chưa xác thực; job lập chỉ mục chỉ nằm trong bộ nhớ.
- Chưa có bộ nhớ dài hạn.

12.3. Hướng phát triển

- Streaming câu trả lời qua SSE; hàng đợi job bền vững; xác thực nhiều người dùng.
- Dùng pgvector để lưu vector thay vì pickle; bộ nhớ dài hạn theo người dùng.
- Mở rộng sang sinh nội dung từ các câu trả lời có trích dẫn, ví dụ tạo slide tóm tắt.
- Mở rộng bộ đánh giá sang nhiều tài liệu và nhiều lĩnh vực.

TÀI LIỆU THAM KHẢO

[1] Sarthi P. et al., “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval”, ICLR 2024.

[2] Yan S.-Q. et al., “Corrective Retrieval Augmented Generation”, 2024.

[3] Lewis P. et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, NeurIPS 2020.

[4] Chen J. et al., “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings”, 2024.

[5] Tài liệu LangGraph: https://langchain-ai.github.io/langgraph/

[6] SQLAlchemy: https://docs.sqlalchemy.org/ ; Streamlit: https://docs.streamlit.io/

[7] Bài báo AgenticRAG (arXiv:2605.05538) được nêu trong tài liệu thiết kế của dự án.